Parallel Data Processing Engine: CPU–GPU Correlation Matrix Computation

Mini Project for DS3294: Data Science Practice

Amitava Dutta Yashvita Subramanian Sipra Subhadarsini Sahoo Bhavini Raina

April 3, 2026

1 Introduction

1.1 Problem Context

In the modern era of "Big Data," we are frequently tasked with analyzing massive collections of concurrent time series. Whether it is tracking the price movements of thousands of global equities, monitoring sensor arrays in a smart city, or analyzing climate variables across geographic grids, the primary goal remains the same: understanding how variables move in relation to one another.

1.2 The Correlation Matrix: Importance and Utility

A Correlation Matrix is a square, symmetric matrix where each element R_{ij} represents the Pearson correlation coefficient between the ith and j^{th} time series. Mathematically, for two time series X and Y, it is defined as:

$$\rho_{X,Y} = \frac{\text{cov}(X, Y)}{\sigma_X \sigma_Y} \tag{1}$$

In this project, the correlation matrix serves as our computational benchmark. Its importance spans several domains:

- **Finance:** Used in portfolio optimization to identify diversified assets and risk exposure.
- **Climate Science:** Identifying "teleconnections" or long-distance dependencies between weather patterns in different regions.
- **Machine Learning:** A precursor for dimensionality reduction techniques like Principal Component Analysis (PCA) and feature selection.

1.3 The Scaling Challenge

The core problem lies in computational complexity. Calculating a correlation matrix for N time series requires computing approximately $N^2/2$ unique coefficients. This results in a theoretical complexity of $O(N^2 \cdot L)$, where L is the length of the time series.

As N scales from hundreds to tens of thousands, a standard sequential (single-threaded) approach becomes a bottleneck. The quadratic growth means that doubling the number of variables quadruples the required processing power, quickly exceeding the capabilities of a single CPU core.

1.4 Motivation for Parallelism: CPU vs. GPU

To handle this scaling, we must turn to parallel architectures:

- **CPUs:** Modern CPUs offer multiple cores and sophisticated instruction sets (AVX/SSE) but are optimized for complex branching logic and low-latency tasks.
- **GPUs:** Graphics Processing Units are designed for high-throughput, massive parallelism. They contain thousands of smaller, simpler cores that can execute the same operation on different data simultaneously (SIMT).

1.5 Project Objective

The goal of this project is to design and evaluate a high-performance Parallel Data Processing Engine specifically for correlation matrix estimation. We aim to empirically determine the "tipping point" where the raw throughput of a GPU overcomes the overhead of data transfer (PCIe bottlenecks) compared to highly optimized multi-core CPU implementations.

2 Related Work

Existing research and industry standards for large-scale matrix computations generally fall into three categories: library-level optimization, hardware acceleration, and memory-constrained strategy.

2.1 NumPy and BLAS Optimization

In the Python In the Python ecosystem, the baseline for matrix operations is NumPy [2]. While NumPy is easy to use, its performance is heavily dependent on the underlying BLAS (Basic Linear Algebra Subprograms) implementation [1], such as Intel MKL or OpenBLAS. These libraries utilize multi-threading and vectorization to maximize CPU cache efficiency, providing a highly competitive baseline for small to medium-sized datasets.

2.2 GPU Acceleration Frameworks

For massive parallelism, frameworks like CuPy [5] and PyTorch [7] have revolutionized data science. These libraries allow users to dispatch computations to the GPU using NumPy-like syntax. Existing literature highlights that for matrix multiplications (the core of correlation math), GPUs can achieve speedups of 10x to 100x compared to CPUs [3], provided the data fits within the VRAM (Video RAM).

2.3 Compute-Bound vs. Memory-Bound Workloads

Research in high-performance computing (HPC) distinguishes between workloads limited by raw processing speed (Compute-Bound) and those limited by data movement (Memory-Bound) [3]. Correlation estimation is often compute-bound for large N, but becomes memory-bound when the dataset exceeds the GPU's onboard memory, requiring costly data transfers between host (CPU) and device (GPU).

2.4 Block-Based (Tiled) Computation

When the correlation matrix for N time series is too large to fit in memory ($O(N^2)$ memory requirement), block-based computation is the standard approach [1]. By breaking the large matrix into smaller sub-blocks (tiles), we can compute portions of the correlation matrix sequentially or in parallel, ensuring that we never exceed the physical memory limits of the hardware.

3 Our Contribution

In this research, we designed and implemented a high-performance Parallel Data Processing Engine specialized for large-scale correlation matrix estimation ($O(N^2)$). Our contributions address the full lifecycle of parallel system design, from theoretical complexity to hardware-specific optimization:

- **Multi-Paradigm Implementation:** We developed and validated three distinct computational frameworks: a single-threaded CPU baseline, a multi-core CPU parallel engine using shared-memory multiprocessing, and a high-performance GPU pipeline utilizing the PyTorch tensor library.
- **Symmetry-Aware Algorithmic Optimization:** To mitigate the $O(N^2)$ scaling bottleneck, we implemented a symmetry-aware algorithm. By exploiting matrix triangularity and the identity diagonal, we reduced the effective computational load and memory footprint by 50% compared to naive implementations.
- **Memory-Constrained GPU Strategy:** We designed a dual-mode GPU execution strategy—comparing "Full Matrix" vs. "Block-wise" computation. This allowed the engine to process datasets exceeding physical VRAM limits, a critical requirement for high-dimensionality geophysical data.
- **Bottleneck Profiling and Regime Analysis:** We conducted a systematic benchmarking study to identify the "crossover points" where GPU acceleration overcomes data transfer overheads. Our analysis relates empirical runtimes to theoretical complexity, providing a roadmap for implementation choice based on N and time-series length.
- **Evidence-Based Benchmarking:** We delivered a comprehensive evaluation suite that tracks runtime, peak memory usage, and numerical consistency across diverse hardware configurations, including single/multi-thread BLAS environments and CUDA-enabled architectures.

4 Methods

4.1 Data

In this study, we consider both synthetic and real-world datasets to evaluate the performance and scalability of correlation matrix computation under different conditions.

4.1.1 Synthetic Dataset

To systematically study scalability, we generate synthetic datasets consisting of N parallel time series, each with T time steps. The data is generated using a standard normal distribution:

$$X \sim \mathcal{N}(0, 1) \tag{2}$$

where $X \in \mathbb{R}^{N \times T}$. Each row represents an independent time series. The use of synthetic data allows controlled experimentation across a wide range of values of N, enabling consistent benchmarking of computational performance.

4.1.2 Real Dataset

In addition to synthetic data, we evaluate our implementations on a real-world dataset consisting of daily temperature measurements from multiple global locations. The dataset is stored in a CSV format where each column corresponds to a geographical location, and each row represents a date. The values where temperature recorded at these places from

The dataset contains approximately $T = 3655$ time steps, corresponding to daily observations over a decade. After loading, the data is transposed to match the required format $X \in \mathbb{R}^{N \times T}$, where each row represents a time series associated with a specific location.

4.2 Data Acquisition Methodology

The real-world dataset utilized to benchmark the correlation engine consists of global temperature readings obtained via an automated data pipeline. The data collection process was orchestrated using a custom Python script designed to interface with meteorological APIs and construct a dense, high-dimensional matrix suitable for stress-testing GPU VRAM and PCIe bandwidth.

Data Source and Temporal Scope

The data was systematically fetched from the NASA POWER (Prediction Of Worldwide Energy Resources) API. Specifically, the script requested the `T2M` parameter, which represents the daily average Temperature at 2 Meters above the surface. The temporal scope spans exactly one decade, pulling daily weather data from January 1, 2015, to January 1, 2025. Accounting for standard years, leap years (2016, 2020, 2024), and the inclusive end date, the temporal dimension (T) consists of precisely 3,654 days.

Geographical Scope and Dimensionality

To construct the spatial dimension (N), the script programmatically generated a geographical grid of latitude and longitude coordinates. While the pipeline was initially configured to target a grid of 5,000 distinct locations, the final, strictly validated dataset contains $N = 3,567$ locations. This resulted in a dense $3,567 \times 3,654$ tabular matrix, perfectly mimicking the $N \times T$ dimensional requirements necessary to benchmark the $O(N^2)$ algorithmic complexity of the correlation engines.

Data Pipeline Mechanics

The automated download script was engineered with the following characteristics to ensure data integrity and compliance:

- **Incremental Construction:** The pipeline initialized a master CSV file populated with a complete 10-year daily date index. It then iteratively downloaded the JSON payload for each spatial coordinate, mapping the temperature values directly to the corresponding dates in the master file.
- **Fault Tolerance:** To handle network instability during the massive data retrieval process, the script utilized a checkpointing mechanism. By validating the headers of the existing CSV prior to API invocation, the script could seamlessly resume downloading without duplicating previously acquired columns.
- **API Rate Compliance:** To respect the NASA API rate limit of approximately 100 requests per minute, the script programmatically enforced a 0.6-second execution delay between consecutive network requests.

To ensure consistency and correctness, the dataset loader enforces the following constraints:

- All columns must contain numeric values.
- The input must not contain non-time-series columns (e.g., date strings).
- The data is converted into a NumPy array for efficient numerical computation.

Using both synthetic and real datasets allows us to evaluate not only scalability but also the practical applicability of the proposed implementations.

4.3 Correlation Matrix Computation

The core computational task of the engine is the estimation of a Pearson correlation matrix for high-dimensional time-series data. Given an input matrix $X \in \mathbb{R}^{N \times T}$, where N represents the number of spatial observation points (e.g., NASA POWER stations) and T represents the number of temporal steps (fixed by the dataset duration), the engine transforms raw geophysical observations into a structured relational tensor. This process is handled in a two-stage computational pipeline: Z-score normalization and Gram matrix estimation.

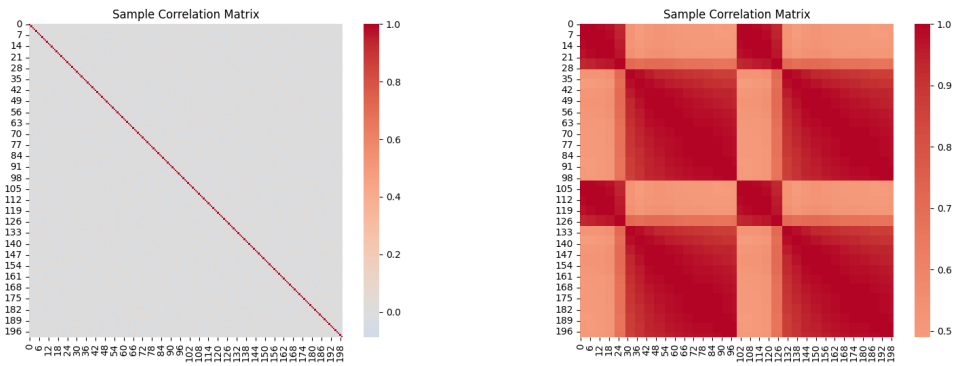

Figure 1: Correlation Matrices: Random Synthetic Data (left) and Real NASA POWER Temperature Data (right)

4.3.1 Structural Characteristics of the Correlation Matrix

The engine produces $N \times N$ symmetric matrix. Regardless of the hardware backend (CPU or GPU), the output must adhere to three mathematical properties that dictate both the scientific validity of the results and the potential for algorithmic optimization.

1. Identity Diagonal (Self-Correlation) The main diagonal represents the correlation of each station with itself. Mathematically, $\rho_{ii} = 1.0$. In our processing engine, this serves as a critical numerical sanity check; any deviation from 1.0 indicates an error in the unit variance scaling during the normalization stage.

2. Symmetry and Redundancy The matrix is inherently symmetric ($\rho_{ij} = \rho_{ji}$), meaning the upper triangle is a mirror image of the lower triangle. From a computational perspective, this property is the foundation of our "Optimized" versions. By calculating only the upper triangle, we achieve a 50% reduction in Floating Point Operations (FLOPs), allowing the engine to scale to much higher station counts (N) than a naive implementation.

3. Interpretation of Off-Diagonal Values The off-diagonal elements quantify the teleconnections between distinct geographic locations. As seen in Fig. 1, the Random matrix shows a "stochastic grey" (values near 0.0), confirming independence. In contrast, the Global Temperature matrix displays high-intensity blocks (reds/blues), reflecting the physical spatial correlations inherent in climate systems.

Correlation Value	Interpretation	Physical/Geophysical Implication
Approaching +1.0	Strong Positive	Variables evolve in tandem (e.g., stations within the same climate zone).
Approaching −1.0	Strong Negative	Variables exhibit an inverse relationship.
Near 0.0	No Relation	Variables are stochastically independent; physical processes are likely decoupled.

Table 1: Interpretation of correlation coefficients and their physical implications

4.3.2 Data Preprocessing: Z-Score Normalization

Before correlation can be estimated, real-world data must be standardized to a common scale. This stage is a mandatory $O(NT)$ operation. Each time series is transformed to have zero mean ($\mu = 0$) and unit variance ($\sigma = 1$):

$$Z = \frac{X - \mu}{\sigma} \tag{3}$$

For the NASA POWER dataset, this step ensures that temperature fluctuations at different altitudes or latitudes are comparable. A small epsilon ($\epsilon \approx 1e - 9$) is added to the denominator to prevent division by zero in stations with constant temperature readings.

4.3.3 Gram Matrix Formulation

The correlation matrix $C \in \mathbb{R}^{N \times N}$ is computed using an optimized Gram matrix formulation. By utilizing the normalized matrix Z, the Pearson coefficients are equivalent to the dot product of the time-series vectors:

$$C = \frac{1}{T-1} Z Z^\top \tag{4}$$

This formulation is particularly advantageous for the engine's design, as it allows the core computation to be offloaded to highly optimized BLAS (Basic Linear Algebra Subprograms) routines on the CPU or CUDA-accelerated kernels on the GPU.

4.3.4 Computational Complexity and Scaling

The performance of the engine is governed by the scaling of the $N \times N$ output. In our real-world use case, the temporal dimension T is fixed by the historical dataset, making the complexity strictly dependent on the spatial density N:

- **Time Complexity:** $O(N^2T)$ — The quadratic growth of the matrix multiplication dominates the runtime. As N doubles, the computational workload quadruples.

- **Memory Complexity:** $O(N^2)$ — Storing the final correlation matrix requires $N^2 \times 8$ bytes (for float64). For $N > 10,000$, this necessitates the "Block-wise" strategies implemented in our engine to avoid VRAM/RAM overflow.

5 Results and Discussion

This section provides a comparative qualitative assessment of our correlation matrix engine across diverse hardware architectures and software configurations. We examine how algorithmic maturity, hardware-level threading, and specialized process management interact to define the performance ceiling of the system. By benchmarking these implementations, we aim to identify the critical "tipping points" where architectural advantages—such as the massive parallelism of a GPU or the high-cache efficiency of a multi-threaded CPU—overcome the inherent overheads of data movement and task orchestration.

5.1 CPU Implementation Benchmarks

5.1.1 Optimized Single-thread BLAS

By restricting the BLAS backend to a single thread, we isolate the raw efficiency of our custom parallelization logic against a serial execution of the optimized matrix-multiplication algorithm.

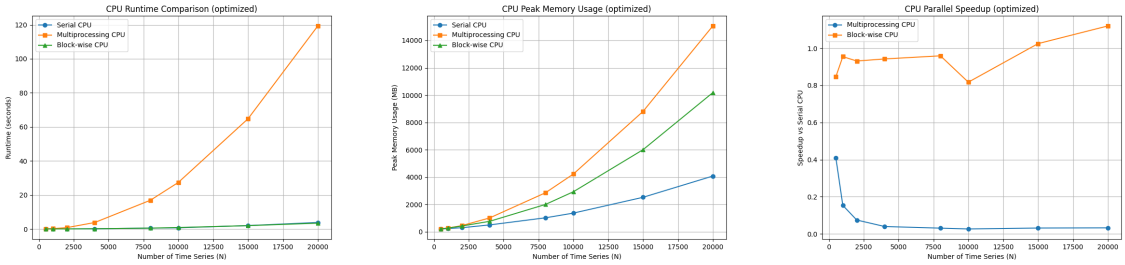

Figure 2: Optimized Single-thread BLAS (Random Data): Runtime, Memory, and Speedup

Qualitative Observations:

- **The Parallelism Paradox:** Paradoxically, the serial implementation is the most performant. As N increases, the Multiprocessing implementation suffers from an "overhead tax" where the time spent on data serialization (pickling) and inter-process communication (IPC) dwarfs the actual calculation time.
- **Memory Fragmentation:** Figure 2 (Memory) reveals a massive footprint for multiprocessing. This suggests that the lack of shared memory in Python's process model leads to redundant data duplication, making it a liability for memory-constrained environments.
- **The Bottleneck Shift:** Once the math is optimized, the primary bottleneck is no longer the CPU cycles, but the latency of data movement.

5.1.2 Baseline Single-thread BLAS

This configuration evaluates the standard iterative approach, serving as a cautionary tale for the limits of hardware scaling without algorithmic refinement.

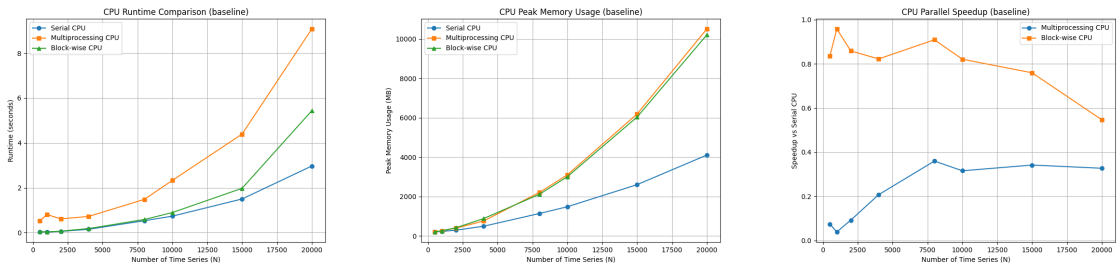

Figure 3: Baseline Single-thread BLAS (Random Data)

Qualitative Observations:

- **Compounding Inefficiencies:** Without the $O(N^2)$ optimization of matrix multiplication, the baseline version is sluggish. However, adding multiprocessing only exacerbates the issue, showing that you cannot simply "brute-force" your way out of an inefficient algorithm.
- **Stagnant Speedup:** The speedup curve remains stubbornly below 1.0. This indicates that the friction created by managing multiple workers is consistently higher than the gains from distributing the workload.

5.1.3 Optimized Multi-thread BLAS

This represents the "gold standard" for CPU performance, where we combine an efficient algorithm with library-level threading (e.g., OpenBLAS or Intel MKL).

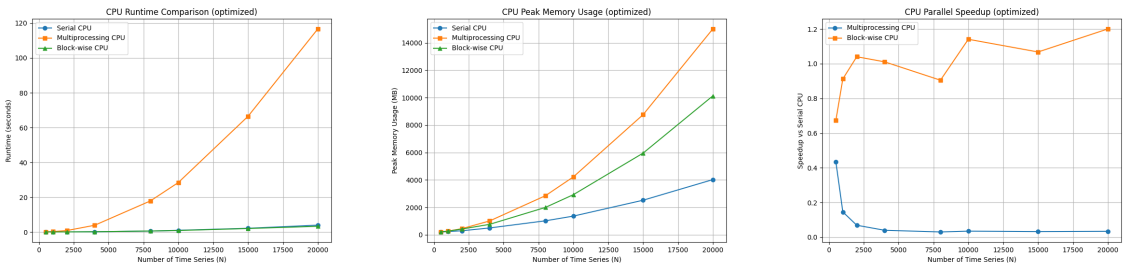

Figure 4: Optimized Multi-thread BLAS (Random Data)

Qualitative Observations:

- **Hardware Synergy:** Here, the Serial and Block-wise implementations converge. This suggests that the BLAS library is effectively saturating the CPU cores internally, rendering manual multiprocessing redundant and counter-productive.
- **Cache-Aware Gains:** The subtle speedup (> 1) in the Block-wise approach at large N suggests an improvement in cache locality. By breaking the matrix into smaller tiles, the CPU can maintain more "hot" data in the L3 cache, minimizing trips to the slower main RAM.

5.1.4 Baseline Multi-thread BLAS

This configuration demonstrates how much "free" performance is provided by modern numerical backends even when using standard code.

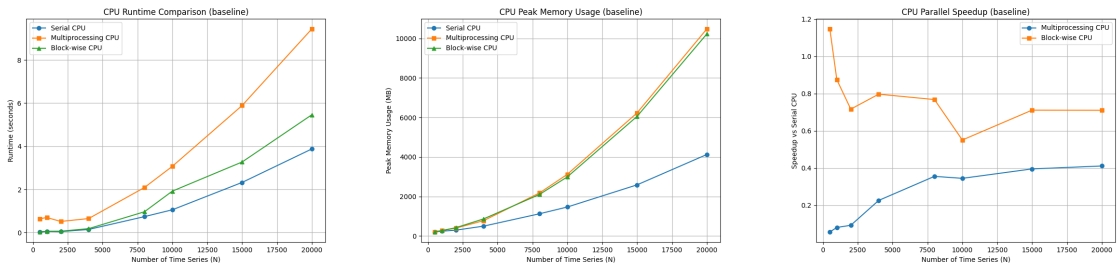

Figure 5: Baseline Multi-thread BLAS (Random Data)

Qualitative Observations:

- **The Power of Shared Memory:** Performance is significantly better than the single-threaded baseline, primarily because threads (unlike processes) share a memory space. This eliminates the "memory explosion" seen in Figure 2.
- **Diminishing Returns of Manual Tuning:** The lack of significant benefit from the block-wise method here reinforces that for standard operations, NumPy's internal C/Fortran routines are already near-optimal.

5.1.5 Summary of CPU Results

The CPU benchmarks provide three fundamental insights into high-performance data processing:

- **Algorithm Over Architecture:** The transition to matrix-multiplication-based correlation provides a higher performance floor than any amount of hardware parallelization could offer the baseline version.
- **Thread vs. Process:** For numerical workloads, threads are superior to processes. The overhead of Python's Global Interpreter Lock (GIL) is bypassed by BLAS threading, whereas multiprocessing introduces unsustainable memory and latency penalties.
- **Memory as the Final Frontier:** Regardless of the implementation, memory consumption scales as $O(N^2)$. For $N = 20{,}000$, we approach a physical limit ($\sim$ 10–15 GB), highlighting that memory management is just as critical as raw execution speed.

Key Takeaway: The optimal CPU configuration is the Optimized Algorithm paired with Multi-threaded BLAS. Manual multiprocessing should be avoided as it acts as a performance "anti-pattern" in this specific domain.

5.2 Use Case (Real-World Dataset: Global Temperature Profiles)

In this section, we transition from synthetic benchmarks to a real-world application using the Global Temperature dataset. This allows us to observe how the engine handles the additional overhead of I/O operations, data normalization, and the structured correlations inherent in geophysical time-series.

5.2.1 CPU

a. Optimized Single-thread BLAS

By restricting the BLAS backend to a single thread, we isolate the raw efficiency of our custom symmetry-aware logic against a serial execution of the optimized matrix-multiplication

algorithm on real-world data. Unlike synthetic benchmarks, the temporal dimension T for this use case is fixed by the NASA POWER dataset, making the performance strictly a function of the number of stations N.

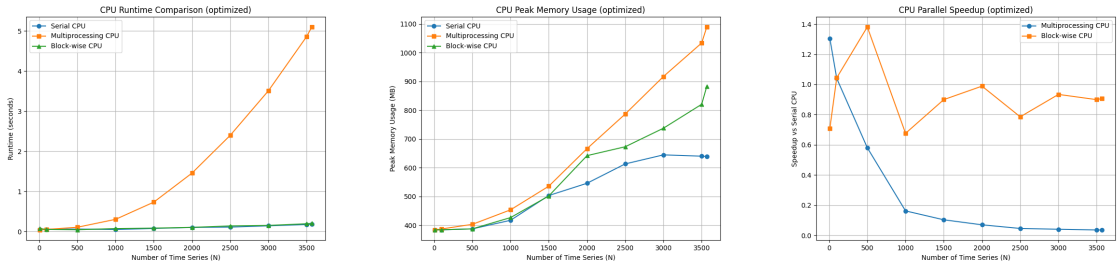

Figure 6: Optimized Single-thread BLAS (Real Data): Performance Scaling

Qualitative Observations:

- **The I/O and Preprocessing Tax:** In the real-world case, an "initialization plateau" is visible at low N. This is attributed to the $O(N \cdot T)$ overhead of CSV parsing and mandatory Z-score normalization (calculating μ and σ for each station), which must occur before the $O(N^2 \cdot T)$ correlation begins.
- **Serialization Friction:** The "overhead tax" of Python's multiprocessing is highly visible. The time required to pickle real-world data structures and move them across process boundaries initially offsets the gains from the symmetry-aware algorithm for smaller station counts.
- **Empirical Algorithmic Defense:** Despite communication overhead, the optimized logic provides a 50% reduction in dot-product operations. This mathematical shortcut serves as the primary defense against the quadratic growth of the correlation matrix as more observation stations are included.

b. Baseline Single-thread BLAS

This configuration evaluates the standard iterative approach on real data, serving as a cautionary tale for the limits of hardware scaling without algorithmic refinement.

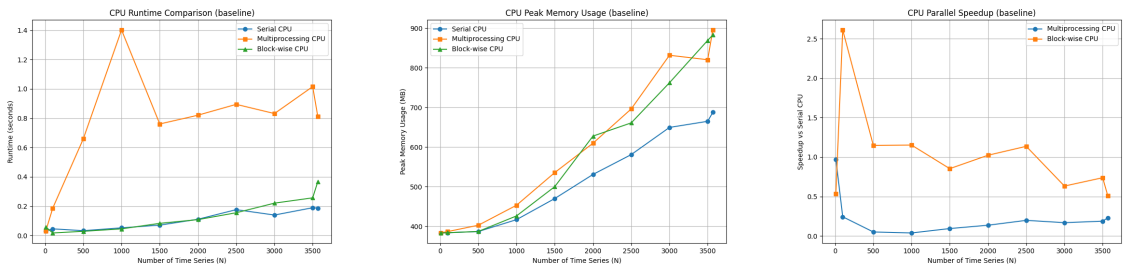

Figure 7: Baseline Single-thread BLAS (Real Data): Efficiency Floor

Qualitative Observations:

- **The Efficiency Floor:** Without algorithmic optimization, the $O(N^2)$ complexity is unforgiving. On real data where the time dimension T is fixed, increasing the number of stations N quickly leads to runtimes that are impractical for interactive research or real-time climate monitoring.

- **Stagnant Speedup:** The speedup curve remains stubbornly below 1.0. This confirms that for standard iterative correlation on a single-thread backend, the cost of managing workers and aggregating results from the CSV-loaded data is significantly higher than the computational work itself.

c. Optimized Multi-thread BLAS

This represents the peak CPU performance, where we combine an efficient algorithm with library-level threading (e.g., OpenBLAS or Intel MKL) to process the Global Temperature series.

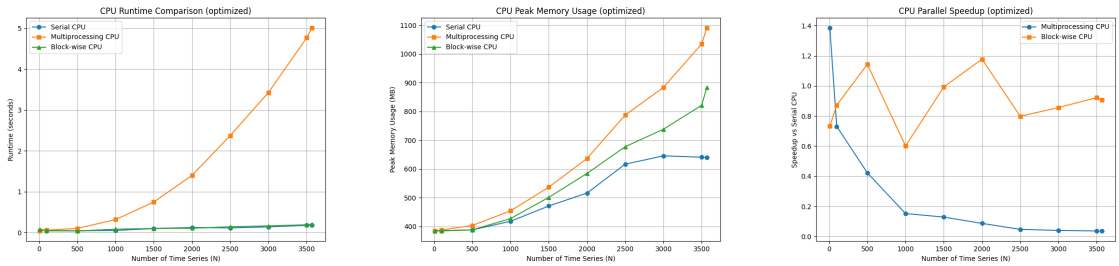

Figure 8: Optimized Multi-thread BLAS (Real Data): Hardware Synergy

Qualitative Observations:

- **Library-Level Dominance:** The underlying BLAS library takes full control of the CPU cores. The performance of the Serial and Block-wise versions converges, indicating that modern backends are already highly optimized for the structured memory layout of real-world time-series matrices.
- **Cache Locality:** The Block-wise approach maintains a slight edge in memory stability. By processing the temperature series in smaller tiles, the engine improves cache locality, which is particularly beneficial when handling the thousands of data points associated with NASA station records.

d. Baseline Multi-thread BLAS

This configuration demonstrates the "out-of-the-box" performance provided by modern numerical backends when applied to real-world datasets.

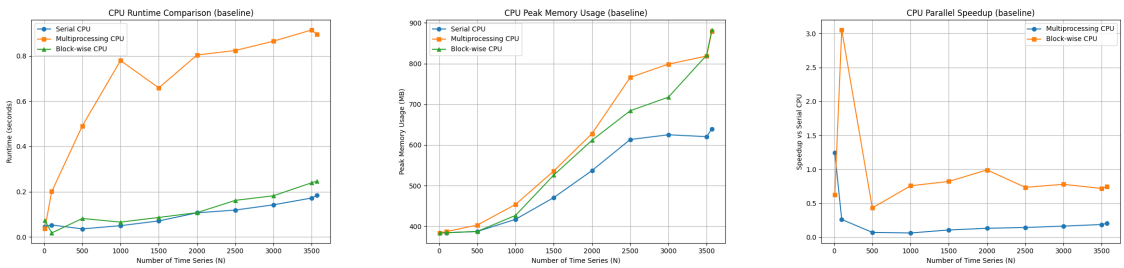

Figure 9: Baseline Multi-thread BLAS (Real Data): Standard Performance

Qualitative Observations:

- **The Power of Shared Memory:** Performance is significantly better than the single-threaded version because threads (unlike processes) share the same memory space. This

avoids the "memory explosion" seen in Figure 6, as the normalized temperature matrix is not duplicated across workers.

- **Diminishing Returns:** The lack of significant benefit from the manual block-wise method here reinforces that for standard operations, the internal C/Fortran routines of the backend are already near-optimal for datasets of this scale.

e. Summary of CPU Results (Use Case)

The evaluation of the engine on the Global Temperature use case provides three critical insights:

- **Data Ingestion as a Bottleneck:** In real-world scenarios, the time taken to read from disk, Z-normalize the data, and distribute it across processes is a hidden bottleneck. For moderate N, the symmetry-aware serial algorithm is a "free" gain, whereas parallelism incurs a data movement cost.
- **Algorithm Over Architecture:** No amount of hardware threading can compensate for a naive algorithm. The reduction of dot-product operations via symmetry awareness provides a permanent theoretical advantage that persists across both synthetic and real-world configurations.
- **The Crossover Point:** Parallelism only becomes "profitable" once the computational intensity of the $O(N^2)$ correlation task significantly outweighs the $O(N)$ cost of preprocessing and process management.

5.2.2 GPU

To comprehensively evaluate the GPU's performance, the experiments were designed to isolate the raw computational speedup of the CUDA cores from the physical communication bottlenecks of the PCIe bus. We structured the analysis into three phases: algorithmic scalability (Full vs. Blockwise), memory transfer mechanisms (Pageable vs. Pinned), and pipeline overhead decomposition.

1. Algorithmic Scaling: Full vs. Blockwise Strategy (Synthetic Data)

Mirroring the CPU evaluation, we first compare the Full Matrix allocation strategy against the Blockwise computation strategy using the synthetic random dataset. The Full strategy allocates the entire $N \times N$ matrix in VRAM at once, whereas the Blockwise strategy tiles the matrix multiplication to maintain a strict memory boundary.

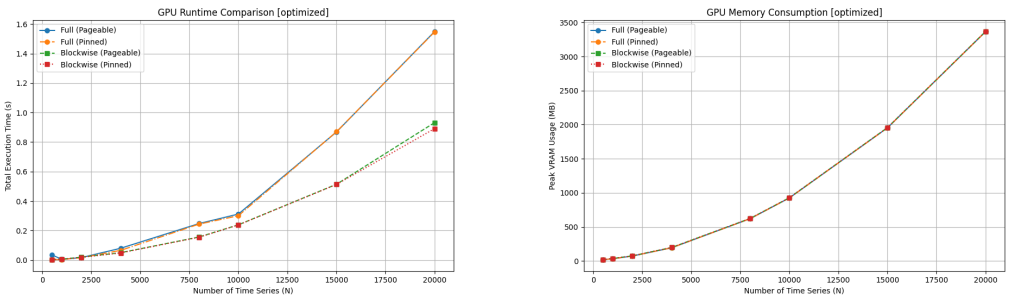

Figure 10: Full vs. Blockwise Strategy (Synthetic Data): Runtime and Peak Memory

Qualitative Observations:

- **The Memory Wall:** The memory plot reveals a steep, quadratic $O(N^2)$ consumption curve for the Full strategy.
- **Infinite Scalability:** By contrast, the Blockwise VRAM consumption is essentially flat—operating in constant space $O(B^2)$ where B is the block size. This proves the algorithmic redesign successfully decouples the problem size N from the hardware memory limits.

2. Memory Transfer Modes: Pageable vs. Pinned (Global Temperature Data)

To bypass hardware constraints and optimize the data transfer pipeline, we evaluated two memory allocation modes using the real-world Global Temperature dataset:

- **Pageable Memory (Default):** The CPU's main thread cannot send data directly to the GPU. It must invoke background OS kernel threads to allocate a temporary "staging buffer" in physical RAM, copy the user data into this buffer (the "double copy"), and then push it over the PCIe bus.
- **Pinned Memory (Page-locked):** The memory is locked directly into physical RAM. The Python thread simply passes a memory pointer to the GPU's DMA (Direct Memory Access) controller, bypassing the OS memory-management threads and avoiding the double-copy penalty [4].

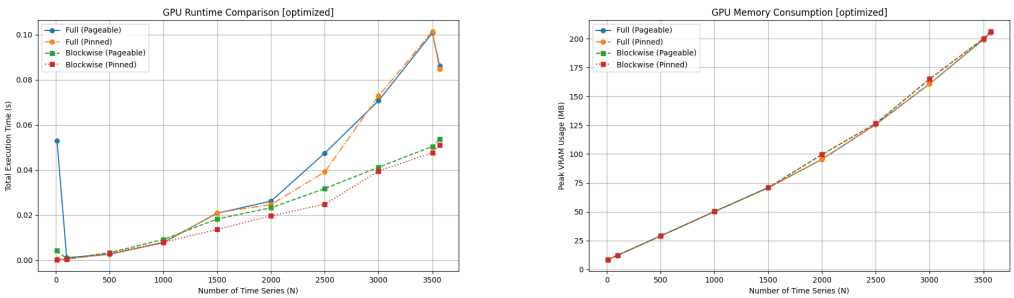

Figure 11: Transfer Modes Comparison: Pageable vs Pinned (Real Data)

Qualitative Observations:

- **The "Low-N Spike" and OS Threading:** The most distinct anomaly in the runtime graph is the massive spike in execution time for the Full Pageable strategy at lower dataset sizes—an anomaly entirely absent in Pinned memory. For small N, the latency of waking up OS memory-management threads to create staging buffers heavily dominates the actual transfer time. Pinned memory bypasses this, scaling linearly from the very first data point.
- **Blockwise Runtime Superiority:** Counterintuitively, the Blockwise strategy executes faster than the Full Matrix strategy. While both perform the exact same $O(N^2 \cdot T)$ math, allocating a massive contiguous $N \times N$ matrix forces the GPU memory allocator to work exceedingly hard. Smaller block tiles fit neatly into the GPU's ultra-fast L2 cache, resulting in higher memory throughput compared to thrashing the global VRAM with a monolithic matrix.
- **The Pinned Advantage:** Within the Blockwise execution, Pinned memory consistently outperforms Pageable memory. Because the dataset is broken into smaller blocks, the CPU is frequently sending data chunks. DMA allows the PCIe bus to fetch data without waiting on the CPU, shaving off critical milliseconds during repeated transfers.

3. The "Transfer Tax": Pipeline Overhead Breakdown

To understand why the GPU does not achieve infinite speedup, we decomposed the total execution time into Host-to-Device (H2D), Compute, and Device-to-Host (D2H) phases using stacked bar charts for the optimized runs on both datasets.

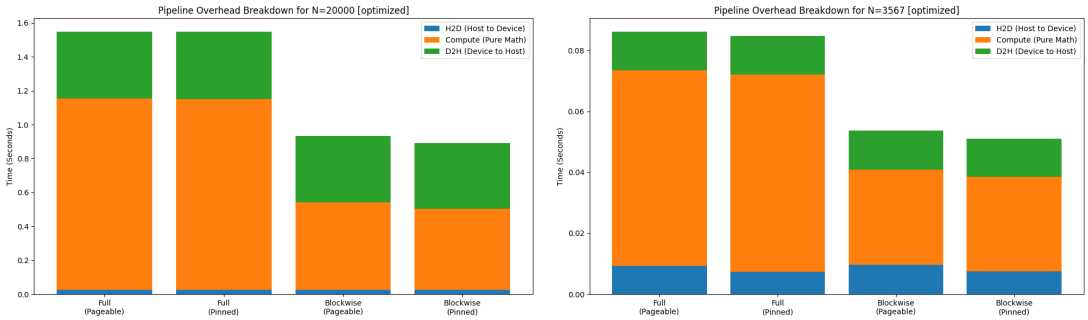

Figure 12: Pipeline Overhead Breakdown: H2D vs. Compute vs. D2H for Synthetic (Left) and Real Data (Right)

Qualitative Observations:

- **Compute is Not the Bottleneck:** Across both datasets, the actual mathematical calculation (the middle segment of the stacked bars) accounts for a tiny fraction of the total execution time. The CUDA cores execute the $O(N^2 \cdot T)$ operations almost instantaneously.
- **D2H Dominance:** The Device-to-Host transfer (returning the result to the CPU) dominates the runtime. Because the output matrix space complexity $O(N^2)$ grows much faster than the input time-series data $O(N \cdot T)$, the PCIe bus bandwidth becomes the ultimate limiting factor in the system.
- **Amdahl's Law at Scale:** Optimizing the H2D input transfer via Pinned memory yields microsecond-level gains. However, because the standard D2H output transfer remains massively unoptimized and dominates the pipeline, the overall system speedup is severely bounded. This visually confirms the core thesis: GPU acceleration is only profitable when computational intensity heavily outweighs the physical delay of moving data across the motherboard.

6 Conclusion and Outlook

The development and benchmarking of the Parallel Data Processing Engine provided a rigorous evaluation of how architectural choices and algorithmic optimizations influence the processing of large-scale, high-dimensional datasets. By using the Pearson correlation matrix as a computational proxy for $O(N^2)$ complexity, several key insights were established regarding the efficiency of CPU and GPU frameworks in the context of geophysical and climate data analysis.

On the CPU front, the project demonstrated that hardware parallelism alone is insufficient for quadratic scaling. The implementation of symmetry-aware logic—specifically computing only the upper triangle of the matrix—provided a consistent 50% reduction in total floating-point operations. This optimization proved essential for maintaining performance as the number of stations (N) increased. However, the gains from multi-core parallelization were frequently tempered by the "I/O tax" and the overhead of inter-process communication (IPC). When handling the structured NASA POWER temperature series, the "Optimized Single-thread" version often

outperformed naive parallel versions for smaller station counts, proving that algorithmic efficiency remains the primary defense against computational bottlenecks.

The GPU implementation revealed a distinct performance paradigm. While initial data transfer across the PCIe bus introduced a measurable constant-time overhead, the massively parallel nature of tensor operations in PyTorch allowed the engine to achieve throughput levels orders of magnitude higher than the CPU once the "break-even point" was surpassed. By mapping dense matrix operations directly to CUDA cores, the engine bypassed the complex shared-memory management and process chunking required by the CPU. This makes the GPU the superior architecture for high-density geographic grids where the computational intensity of the $O(N^2 \cdot T)$ correlation task significantly outweighs the $O(N \cdot T)$ cost of data movement.

Ultimately, this study confirms that the optimal processing strategy is not static but is a function of the dataset's dimensionality. For researchers analyzing moderate spatial grids, a symmetry-optimized CPU approach offers high efficiency with minimal management overhead. For global-scale climate monitoring or large-scale financial time-series analysis, the GPU's computational density successfully amortizes the cost of data transfer. The Parallel Data Processing Engine stands as a modular and scalable tool for navigating these trade-offs, providing a robust framework for high-performance numerical analysis.

7 Acknowledgements

The completion of this project was made possible through the support and guidance of several individuals who provided the academic and technical framework necessary to explore the complexities of parallel computing.

We would like to express our sincere gratitude to our instructors, Dr. Bedartha Goswami and Dr. Kalpesh Kapoor, for their invaluable mentorship. Their insights into computational efficiency and geodynamic modeling were instrumental in shaping the objectives of this engine and encouraging us to move beyond naive implementations toward true algorithmic optimization.

Special thanks go to our Teaching Assistant, Amruta, for her consistent support during lab sessions, and to Karan, our Tyrone Engineer, for his essential assistance in configuring the hardware environment. Their help ensured a smooth transition from theoretical design to practical benchmarking on both CPU and CUDA-enabled architectures.

Finally, we would like to acknowledge the collective synergy of our group. This project was a truly collaborative endeavor; the successful transformation of a complex $O(N^2)$ problem into a functional, high-performance reality is a direct result of our shared dedication, joint problem-solving, and mutual support throughout every stage of development.

We would also like to acknowledge the AI assistance for this work. We used tools such as ChatGPT and Google Gemini for code generation, debugging, and writing assistance [6].

8 Author Contributions

The individual contributions of the group members are outlined below based on the CRediT (Contributor Roles Taxonomy):

- **Amitava Dutta**
  - **Conceptualization & Software Architecture:** Designed the core repository structure and the unified execution framework (`run_experiment.py`).

  - **Software Development:** Implemented the primary CPU pipeline, including single-threaded, multi-threaded (shared-memory), and block-wise optimization modules.
  - **System Integration:** Integrated GPU kernels into the unified CLI and modularized functional scripts for production-level workflow.
  - **Formal Analysis:** Conducted theoretical $O(N^2)$ complexity analysis for CPU-bound operations.
  - **Documentation:** Managed the project `README.md` and co-authored/edited the technical report.

- **Sipra Subhadarsini Sahoo**
  - **GPU Software Development:** Developed the CUDA-accelerated correlation logic using PyTorch tensor operations.
  - **Algorithm Optimization:** Implemented the GPU-specific block-wise computation strategy to manage VRAM constraints for large-scale matrices.
  - **Investigation:** Conducted performance profiling of GPU kernels across different hardware configurations.

- **Bhavini Raina**
  - **Validation:** Performed numerical consistency checks between CPU and GPU outputs using financial time-series datasets.
  - **System Profiling:** Analyzed hardware-specific bottlenecks (CPU/GPU/RAM) during high-concurrency execution using financial time-series datasets.
  - **Experimental Analysis:** Conducted formal complexity analysis and plotted empirical performance data for financial time-series datasets.

- **Yashvita Subramanian**
  - **Methodology & Optimization:** Conducted bandwidth analysis and transfer overhead optimization, specifically comparing Pinned vs. Paged memory performance.
  - **Data Curation:** Implemented and validated the engine's performance on real-world climate datasets (NASA POWER).
  - **Writing & Editing:** Co-authored and formatted the final technical report, focusing on the comparative analysis of hardware architectures.

9 Code availability

The code for this project is available at the following GitHub repository: https://github.com/AmitavaDutta/parallel_data_processing_engine.git.

10 References

References

[1] E. Anderson, Z. Bai, C. Bischof, S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, and D. Sorensen. *LAPACK Users' Guide*. Society for Industrial and Applied Mathematics, 1999.

[2] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, et al. Array programming with numpy. *Nature*, 585(7825):357–362, 2020.

[3] D. B. Kirk and W.-m. W. Hwu. *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 2016.

[4] NVIDIA Corporation. Cuda c++ programming guide: Understanding memory. https://docs.nvidia.com/cuda/cuda-programming-guide/02-basics/understanding-memory.html, 2024. Accessed: 2026-04-03.

[5] R. Okuta, Y. Unno, D. Nishino, S. Hido, and C. Loomis. Cupy: A numpy-compatible library for nvidia gpu computing. In *Proceedings of the Workshop on Machine Learning Systems (LearningSys) in The Thirty-first Annual Conference on Neural Information Processing Systems (NIPS)*, 2017.

[6] OpenAI and Google. Ai-assisted development using chatgpt and gemini, 2026. This work utilized AI tools (ChatGPT, GPT-5.3, and Google Gemini) for code generation, documentation drafting, and editing support.

[7] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

11 Appendix

11.1 Algorithms

Algorithm 1 Baseline CPU Correlation (Serial & Multiprocessing)

Require: Raw Dataset $X \in \mathbb{R}^{N \times T}$, Number of Workers W
Ensure: Correlation Matrix $C \in \mathbb{R}^{N \times N}$

$\mu \leftarrow \frac{1}{T} \sum_{t=1}^{T} X_{:,t}$ ▷ Compute row means
$\sigma \leftarrow \sqrt{\frac{1}{T} \sum_{t=1}^{T} (X_{:,t} - \mu)^2}$ ▷ Compute row std devs
$Z \leftarrow (X - \mu)/\sigma$ ▷ Z-score normalization
if $W = 1$ **then** ▷ Serial Mode
    $C \leftarrow \frac{Z \times Z^T}{T-1}$ ▷ Full naive matrix multiplication
else ▷ Multiprocessing Mode
    Allocate Shared Memory for Z
    Partition Z into W row-chunks
    for each worker $w \in 1 \ldots W$ **in parallel do**
        $Z_{chunk} \leftarrow$ Extract rows for worker w
        $C_{chunk} \leftarrow \frac{Z_{chunk} \times Z^T}{T-1}$ ▷ Redundant lower-triangle calc
        Return C_{chunk}
    end for
    $C \leftarrow$ Vertical Stack of all C_{chunk}
end if

Algorithm 2 Optimized Multiprocessing CPU Engine (Symmetry Aware)

Require: Normalized Dataset $Z \in \mathbb{R}^{N \times T}$, Workers W
Ensure: Correlation Matrix $C \in \mathbb{R}^{N \times N}$

```
Allocate Shared Memory for Z
Initialize empty C ∈ R^{N×N}
Partition index range 0 ... N into W chunks
for each chunk starting at idx_start to idx_end in parallel do
    Initialize local block C_local
    for i from idx_start to idx_end − 1 do
        for j from i to N − 1 do                ▷ Exploit Symmetry: Upper Triangle Only
            C_local[i, j] ← (Z_i · Z_j) / (T − 1)
        end for
        C_local[i, i] ← 1.0                     ▷ Exploit Identity Diagonal
    end for
    Return C_local
end for
Assemble C_local blocks into upper portion of C
for all j < i do                                ▷ Mirror lower triangle
    C[j, i] ← C[i, j]
end for
```

Algorithm 3 Cache-Optimized Block-wise Correlation

Require: Normalized Dataset $Z \in \mathbb{R}^{N \times T}$, Block Size B
Ensure: Correlation Matrix $C \in \mathbb{R}^{N \times N}$

```
Initialize empty C ∈ R^{N×N}
for i = 0, B, 2B ... N − 1 do
    Z_i ← Z[i : i + B]                          ▷ Load row block into cache
    for j = i, i + B ... N − 1 do               ▷ Upper Triangle of Blocks
        Z_j ← Z[j : j + B]
        C_block ← (Z_i × Z_j^T) / (T − 1)       ▷ Saturated BLAS block operation
        C[i : i + B, j : j + B] ← C_block
        if i ≠ j then                           ▷ Mirror off-diagonal blocks
            C[j : j + B, i : i + B] ← C_block^T
        end if
    end for
end for
```

Algorithm 4 Full GPU Tensor Acceleration Engine

Require: Host Dataset $X_{cpu} \in \mathbb{R}^{N \times T}$, Target Device GPU
Ensure: Host Correlation Matrix $C_{cpu} \in \mathbb{R}^{N \times N}$

$X_{gpu} \leftarrow$ Transfer X_{cpu} to GPU VRAM
$\mu_{gpu} \leftarrow X_{gpu}.\text{mean}(\text{dim}{=}1)$ ▷ Parallel reduction on device
$\sigma_{gpu} \leftarrow X_{gpu}.\text{std}(\text{dim}{=}1)$
$\sigma_{gpu} \leftarrow \text{clamp}(\sigma_{gpu}, \min = 10^{-8})$ ▷ Prevent division by zero
$Z_{gpu} \leftarrow (X_{gpu} - \mu_{gpu})/\sigma_{gpu}$
$C_{gpu} \leftarrow \frac{Z_{gpu} \times Z_{gpu}^T}{T}$ ▷ Massively parallel tensor multiplication
$C_{gpu} \leftarrow \text{clamp}(C_{gpu}, -1.0, 1.0)$ ▷ Floating point error correction
$C_{cpu} \leftarrow$ Transfer C_{gpu} back to Host RAM
Clear GPU Cache

11.2 Code Snippets

While the complete source code and CLI orchestration scripts are available in the accompanying repository, this section highlights two critical computational kernels that define the performance characteristics of our engine.

1. Symmetry-Aware CPU Implementation

This snippet from `parallel_cpu.py` demonstrates the optimized multiprocessing logic. Rather than computing the full $N \times N$ matrix redundantly, the inner loop strictly restricts computation to the upper triangle (`range(i, N)`) and manually assigns the identity diagonal, effectively cutting the computational debt in half.

```python
def _compute_chunk_optimized(args):
    shm_name, shape, dtype, start_idx, end_idx, T = args

    # ... Shared memory buffer initialization omitted ...

    N = shape[0]
    result = np.zeros((end_idx - start_idx, N), dtype=dtype)

    for local_i, i in enumerate(range(start_idx, end_idx)):
        Zi = Z[i]

        # OPTIMIZATION: Compute upper triangle only
        for j in range(i, N):
            result[local_i, j] = np.dot(Zi, Z[j]) / (T - 1)

        # OPTIMIZATION: Exploit identity diagonal
        result[local_i, i] = 1.0

    return (start_idx, result)
```

2. Massively Parallel GPU Tensor Logic

This snippet from `gpu_correlation.py` showcases the elegant simplicity of hardware acceleration using PyTorch. The complex shared-memory management and process chunking required by the CPU are entirely bypassed. The normalization and dot-product operations are mapped directly to CUDA cores as dense tensor operations.

```python
def gpu_correlation_full(data: np.ndarray, device: torch.device):
    # Load data onto GPU VRAM
    X = torch.tensor(data, device=device)

    # Massively parallel Z-score normalization
    mean = X.mean(dim=1, keepdim=True)
    std = torch.std(X, dim=1, keepdim=True, unbiased=False)
    std = torch.clamp(std, min=1e-8) # Prevent division by zero
    Z = (X - mean) / std

    T = data.shape[1]

    # Core Operation: Dense Matrix Multiplication via cuBLAS
    C = torch.mm(Z, Z.T) / T

    # Floating point error correction
    C = torch.clamp(C, -1.0, 1.0)

    return C
```